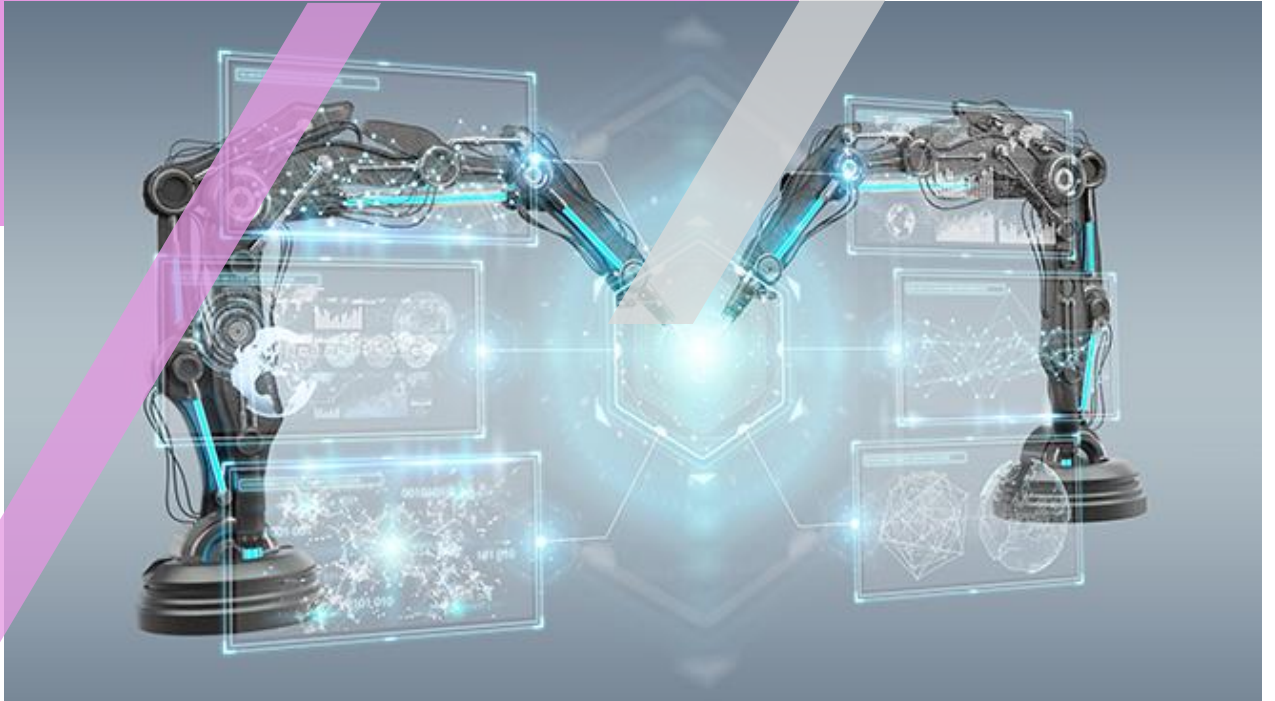


LU3MEE01- Systèmes Robotiques

REALISATION EXPERIMENTALE D'UN ROBOT SERIEL

RAPPORT PROJET



Enseignants : Mr HALIYO Sinan

TABLE DES MATIERES

SEANCE 1 : MODELISATION ET COMMANDE GEOMETRIQUE D'UN ROBOT SERIEL

– Application au 3R plan- -----2

SEANCE 2 : REALISATION EXPERIMENTALE D'UN ROBOT SERIEL -----8

SEANCE 3 ET 4 : REALISATION EXPERIMENTALE D'UN ROBOT SERIEL

-Génération de mouvements et commande -----19

PARTIE BONUS ----- 25

CONCLUSION -----28

SEANCE 1 : MODELISATION ET COMMANDE GEOMETRIQUE

D'UN ROBOT SERIEL

- Application au 3R plan-

Objectif : L'objectif de cette séance est de créer des outils permettant de calculer les modèles géométriques directs et inverses pour un robot plan à 3 degrés de liberté (3R).

2) Modélisation

2.2) Construction et affichage d'un 3R plan

2.2.1) On commence par déclarer le modèle d'un robot 3R plan dans une variable modèle en faisant appel à la fonction robot, déjà définie dans le logiciel de programmation.

```
% Question 2.2.1 : déclaration du modèle r3
modele = robot('model',[0.5 , 0.3, 0.2])
```

2.2.2) On affiche ensuite le modèle à l'aide de la commande modele.plot (), qui nous permet de visualiser le robot par simulation graphique. On vérifie que les positions articulaires ont bien été initialisées à 0.

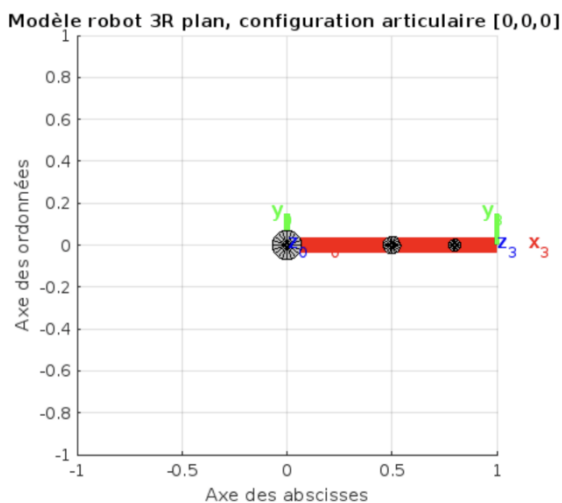
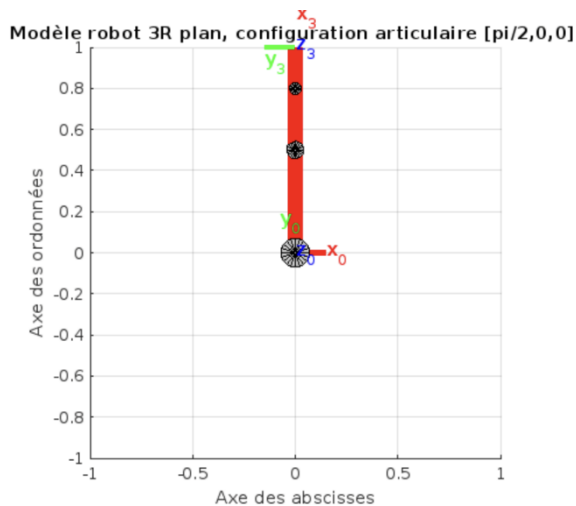


Figure 1- Modèle robot 3R position initiale [0 0 0]

2.2.3) On souhaite maintenant placer le modèle de ce robot dans la configuration articulaire $[\pi/2 \ 0 \ 0]$.



A l'aide de l'affichage graphique, on vérifie bien que la position de l'organe terminal correspond bien à la position attendue.

Figure 2- Modèle robot 3R position initiale $[\pi/2 \ 0 \ 0]$

2.3) Fonctionnalités élémentaires

2.3.1) Nous allons à présent écrire une fonction qui retourne une matrice de rotation d'un angle a autour de l'axe Z. On sauvegarde cette fonction dans un fichier rotation_Z_etu.m

```
function [R] = rotation_Z_etu(a)
% ROTATION_Z retourne la matrice de rotation 3D
% correspondant à une rotation d'angle 'angle' autour de x
% R = rotation_Z(angle)

R = eye(3);
R = [cos(a), -sin(a), 0; ...
     sin(a), cos(a), 0; ...
     0, 0, 1];

end
```

On fait ensuite un appel test à cette fonction $R = \text{rotation_Z_etu}(\pi/2)$ dans le script principal.

R =

```
0.0000    -1.0000         0
1.0000     0.0000         0
0         0         1.0000
```

On obtient ainsi la matrice rotation de l'angle $\pi/2$

2.3.2) Essayons maintenant d'inverser le raisonnement, c'est-à-dire obtenir l'angle de rotation autour de l'axe Z à l'aide de la matrice rotation. De la même manière on écrit une fonction, que l'on sauvegarde dans un fichier `inv_rotation_Z_etu.m`

```
function [a] = inv_rotation_Z_etu(R)
% INV_ROTATION_Z retourne l'angle de rotation autour de l'axe Z à partir
% d'une matrice de rotation passée comme argument (et dont on suppose
% qu'elle représente bien une rotation d'axe Z).
% [a] = inv_rotation_Z(R)

a = atan2 (R(2,1), R(1,1))

end
```

Un appel dans le script principal nous renvoie `a =` ce qui correspond bien à $\pi/2$.

1.5708

2.3.3) La matrice rotation maintenant obtenue, la suite logique est de déterminer la matrice de transformation homogène correspondant à une rotation R et une translation T successives.

On écrit donc la fonction `trans_homogene_etu(R,T)`

```
function [H] = trans_homogene_etu(R,T)
% TRANS_HOMOGENE retourne la matrice de transformation homogene 3D
% correspondant à une rotation R et à une trH =

H = eye(4);
                                0.0000    -1.0000         0     1.0000
                                1.0000     0.0000         0         0
H(1:3, 1:3) = R;                0         0     1.0000         0
H(1:3 , 4)= T;                  0         0         0     1.0000
end
```

2.4) Calcul du modèle géométrique direct et inverse

Nous allons maintenant nous intéresser aux modèles géométriques direct et inverse qui permettent de contrôler le mouvement des robots.

2.4.1) et 2.4.2) Intéressons-nous d'abord au modèle géométrique direct. Il permet de déterminer la position et l'orientation finale d'un robot en fonction des valeurs de ses paramètres articulaires. En d'autres termes, ce modèle permet de calculer les coordonnées du point (x,y,θ) à partir d'une configuration articulaire donnée (q_1, q_2, q_3) .

Commençons par écrire une fonction qui retourne la matrice de transformation homogène ${}^R_0H_{Rfin}$, associés à l'organe terminal du modèle :

```
function THf = mod_geo_dir_etu(modele)
% MOD_GEO_DIR retourne la matrice de transformation homogène associée à
% l'organe terminal du modèle
%THf = mod_geo_dir(modele)

% Vecteur contenant la longueur des segments
long_segment = modele.bodyLength;

% Vecteur contenant les configurations articulaires en rad
conf_artic = modele.getAngularPosition();

for k = length(modele)
    TH = eye(4);

    TH0(1:3, 1:3) = R*TH
    TH0(1:3, 4) = R(1:3,1)*long_segment(k)
    THf = THf*TH0
end
```

On teste cette fonction dans le code principal, en utilisant la fonction `modele.mod_geo_dir()` disponible dans l'annexe afin de comparer nos matrices :

```
THf = mod_geo_dir(modele)
modele.mod_geo_dir
```

En voici un exemple :

THf =		ans =
-0.7071 0.7071 0 -0.4414		-0.7071 0.7071 0 -0.4414
-0.7071 -0.7071 0 0.3586		-0.7071 -0.7071 0 0.3586
0 0 1.0000 0		0 0 1.0000 0
0 0 0 1.0000		0 0 0 1.0000

Les deux matrices sont identiques.

2.4.3) et 2.4.4) Précédemment, nous avons aussi parlé du modèle géométrique inverse. Contrairement au modèle direct, il permet de calculer les valeurs des paramètres articulaires nécessaires pour atteindre une position et une orientation spécifique de l'extrémité du robot. Cette fois, on part donc de la position finale pour trouver les positions articulaires nécessaires pour l'atteindre.

Illustrons ce modèle grâce à la fonction `modele.mod_geo_inv()`, déjà implémentable dans notre programme. On commence d'abord par choisir une configuration cible :

```
cible = [1.0 1.0 3*pi/2]
[q1,q2]=modele.mod_geo_inv(cible(1), cible(2), cible(3))
```

Notons ici un point important, qui nous sera très utile par la suite : cette fonction fournit deux solutions, en raison de la symétrie de l'espace opérationnel. Ainsi, on stocke ces deux solutions dans un vecteur q_1 et q_2 . Il est possible d'obtenir deux solutions identiques, qui indiquent une superposition des vecteurs.

```

cible = [1.0 1.0 3*pi/2]

[q1,q2]=modele.mod_geo_inv(cible(1) , cible(2), cible(3))
modele.setAngularPosition([q1])
modele.plot(1, 'green', 0)
modele.mod_geo_dir
modele.setAngularPosition([q2])
modele.plot(1, 'blue',0)
modele.mod_geo_dir

```

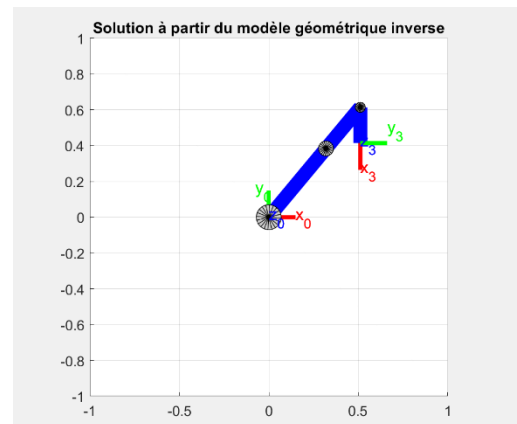


Figure 3- Solution à partir du modèle géométrique inverse

On ne fait pas ici la distinction entre la solution bleue et la solution verte, ce qui indique une superposition des solutions. La solution est donc unique.

3) Génération de trajectoires par interpolation

Lorsque l'on doit placer l'organe terminal d'un bras robotique dans une position spécifique, on doit générer une trajectoire pour chaque moteur du robot. Pour ce faire, on retrouve deux méthodes principales : l'interpolation articulaire et l'interpolation opérationnelle.

3.1) Interpolation articulaire

L'interpolation articulaire consiste à générer une trajectoire en interpolant entre les configurations articulaires du robot. Ainsi, étant donné une configuration articulaire de départ quelconque et une cible opérationnelle (position et orientation dans le plan), nous allons interpoler sur N positions articulaires intermédiaires dans l'espace articulaire.

```

% Question 3.1.1 : interpolation articulaire
qi = [0 0 0];
qf = [pi/2 -pi/2 pi]
N=100;
for n=0:N
    q = qi + n/N .* (qf - qi);
    modele.setAngularPosition(q);
    modele.plot(0, 'red' , 0)
    pause(0.01)
end

```

Les variables q_i et q_f représentent les configurations articulaires initiale et finale du robot. On génère chaque étape intermédiaire de la trajectoire, et à chaque itération de la boucle, la configuration articulaire est calculée en interpolant entre les configurations initiales et finales.

3.2) Interpolation opérationnelle

Concernant l'interpolation opérationnelle, celle-ci consiste à générer une trajectoire en interpolant entre les positions et orientations de l'organe terminal dans l'espace opérationnel. A la différence de l'interpolation articulaire, on ne se concentre pas ici sur les mouvements des articulations individuelles, mais sur la trajectoire de l'organe terminal dans l'espace.

Ainsi, dans une même approche que l'interpolation articulaire, on interpole entre les positions et orientations de l'organe terminal.

```
% Définition des positions et orientations initiale et finale de l'organe terminal
P_initial = [0 0 0]; % Position initiale (coordonnées XYZ)
O_initial = [0 0 0]; % Orientation initiale (angles d'Euler ou quaternion)
P_final = [1 1 1]; % Position finale
O_final = [pi/2 pi/4 pi]; % Orientation finale

% Nombre d'étapes d'interpolation
N = 100;
for n = 0:N
    % Calcul des positions et orientations intermédiaires
    P = P_initial + n/N * (P_final - P_initial);
    O = O_initial + n/N * (O_final - O_initial);

    % Affichage de la position et de l'orientation (optionnel)
    disp(['Étape ' num2str(n) ': Position = ' num2str(P) ', Orientation = ' num2str(O)]);

    % Pause pour visualisation (en pratique, vous enverriez les commandes de mouvement à votre robot)
    pause(0.01);
end
% -----
```

Penchons-nous sur les avantages et les inconvénients de ces deux méthodes :

L'interpolation articulaire permet un contrôle précis des articulations, et prend en compte les limites des articulations du robot. Cependant, elle dépend fortement de la cinématique du robot et ne permet pas un vrai contrôle de l'organe terminal, ce qui implique qu'une simple modification de sa structure doit nécessiter des ajustements de calculs.

L'interpolation opérationnelle permet quant à elle un contrôle direct de l'organe terminal et paraît plus flexible pour différentes configurations. Les calculs nous ont semblé cependant plus complexes puisque cela nécessite une conversion entre les coordonnées de l'espace opérationnel, et les valeurs articulaires du robot.

Bilan : Cette première séance nous aura permis de poser les bases quant à la manipulation de MATLAB, et de nous familiariser avec ses commandes. De plus, on a pu se questionner sur les commandes d'un robot, et sur les concepts fondamentaux en Robotique. Nous pouvons maintenant aller plus loin, et essayer de réaliser un robot type « bras » fonctionnant dans le plan

SEANCE 2 : REALISATION EXPERIMENTALE D'UN ROBOT SERIEL

- Montage, caractérisation et commande des actionneurs-

Objectif : L'objectif de cette séance est de connecter le robot à une alimentation et à son contrôleur, de caractériser chacun des actionneurs du système, et de contrôler en position l'organe terminal du bras.

Avant de nous plonger dans le vif du sujet, faisons une légère présentation de notre système robotique :

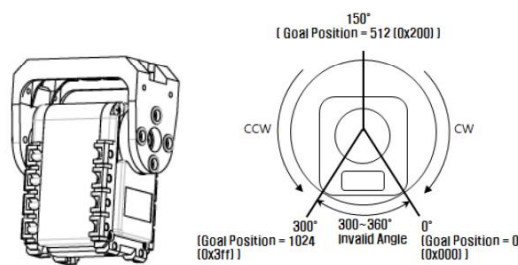


FIGURE 1 – (gauche) Moteur AX12. (droite) Convention pour le repérage des positions angulaires du moteur.

Dans ce projet, nous allons utiliser des moteurs DYNAMIXEL, tous identiques, et repérés par un identifiant (ID), qui nous permettra de communiquer avec chacun des moteurs, de manière indépendante. Les moteurs seront utilisés en mode « position », et vont ainsi réagir à un changement de consigne exprimé en « pas moteur », dont la valeur est absolue et codée en 0 et 1024.

2.1) Prise en main du contrôle des moteurs

On commence par vérifier le zéro moteur, qui correspond à la position 150° sur la figure 1. On connecte ensuite la rallonge USB à l'ordinateur et à l'USB2DYNAMIXEL. Il est ici très important de bien identifier le numéro du port série, afin de permettre la bonne communication avec le robot. Nous entrons ensuite la valeur de l'ID du dernier moteur.

```
% Paramètres de la communications série
PORT = 12;      % A modifier avec le n° du port COM utilisé

%% 2.1 Prise en main du contrôle des moteurs
% =====
ID = 3;         % ID du moteur terminal
maquette = robot('real',PORT,1000000); % Connexion aux moteurs
```

Ces données complètent l'objet maquette (au sens de la programmation), à l'aide de la fonction robot déjà introduite dans le dernier TP. Un set de différentes fonctions sont disponibles directement, et nous serons très utiles dans la suite du TP.

```
maquette = robot('real', port , baud); % Load DLL and create maquette object
maquette.setStepPosition(ID,pos);
maquette.setSpeed(ID,speed);
maquette.isMoving(ID);
maquette.getStepPosition(ID);
maquette.getSpeed(ID);
maquette.close();
```

Nous finalisons cette prise en main, en bougeant le moteur commandant le bout du bras en une position pos quelconque, et en modifiant la valeur de la vitesse de rotation du moteur.

```
% 2.1.6 : utilisation de la fonction setStepPosition
position = maquette.getStepPosition ([ID]) %Renvoie la position articulaire de
                                           %l'actionneur dont l'identifiant
                                           %est indiqué dans la variable ID,
                                           %en pas moteur

pos = 300
maquette.setStepPosition(ID,pos)
maquette.setStepPosition([3,2,1], [800,200, 500])
```

On teste d'abord la commande get.StepPosition(ID), qui nous renvoie, dans la variable position, la position articulaire de l'actionneur souhaité, et ce en pas moteur.

On fait ensuite bouger cette actionneur dans une position pos=300 (en pas moteur). Puis on tente de faire bouger les 3 actionneurs, dans 3 positions différentes. Après plusieurs tests, on observe que le robot se bloque pour certaines valeurs. Nous commençons donc à nous poser la question de limite.

```
% 2.1.7 : utilisation de la fonction setSpeed
speed = maquette.getSpeed(ID) %Renvoie dans la variable speed la valeur de la
                              %vitesse courante de l'actionneur d'ID 3

maquette.setSpeed(ID,200)    %Règle la vitesse de rotation speed de
                              % l'actionneur d'identifiant ID
```

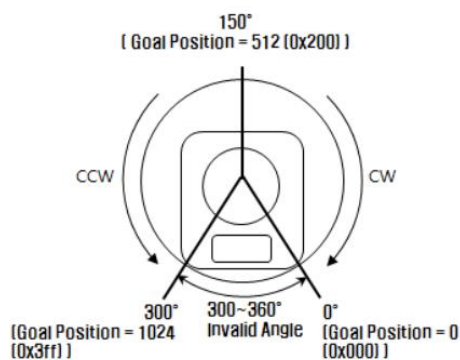
On teste ensuite la fonction get.Speed(ID), qui nous renvoie la vitesse de l'actionneur d'identifiant ID. On modifie ensuite la valeur de cette vitesse à l'aide de la fonction SetSpeed.

2.2) Caractérisation des actionneurs

On va maintenant chercher à caractériser le fonctionnement des actionneurs.

Utilisation du moteur en bout de chaîne cinématique

2.2.8) Nous allons exprimer la relation mathématique qui relie les valeurs des pas moteurs lues depuis l'actionneur (0 à 1024) en valeur angulaire exprimée en radian. Reprenons la figure fournie dans le TP :



On remarque que l'angle parcouru est compris entre 0° et 300°. On appelle 'Invalid Angle' l'angle compris entre 300° et 360°. Ainsi, les positions peuvent aller de 0 à 1024. On appelle r la résolution angulaire et q la valeur angulaire. Offset est la valeur du pas moteur correspondant à l'angle $q=0$ rad (valeur de référence).

On définit d'abord r :
$$r = \frac{300\pi}{180 \times 1024}$$

On écrit ensuite la relation mathématique, qui lie ces variables : $q = -r * (step - offset)$

On peut observer un signe (-) dans la formule. Ceci correspond à une convention pour définir la direction positive.

2.2.9) On peut ainsi écrire une fonction $q = \text{step2angle}(\text{step}, \text{offset})$. On utilise la commande $\text{deg2rad}()$, afin de convertir l'angle en degrés, en radian.

```
function q = step2angle(step,offset)
% STEP2ANGLE convertit la valeur de pas moteur STEP en une valeur angulaire
% Q exprimée en rad.
% OFFSET permet de spécifier à quelle valeur de pas moteur correspond la
% valeur angulaire 0.
%
% q = step2angle(step,offset);
%offset = 512( %moitié de 1024 pour avoir position 0 au centre ([-150°;+150°])

q=0;
r=deg2rad(300)/1024; %Résolution angulaire en rad/pas

q = - r .* (step - offset);
```

Cette fonction convertira donc un step en coordonnées angulaires.

2.2.10) Nous souhaitons maintenant faire le chemin inverse, c'est-à-dire inverser la relation obtenue et relier cette fois un angle exprimée en radian à la valeur de pas moteur correspondante. On initialise de la même manière la résolution angulaire r , et on définit p la valeur de pas moteur.

On s'inspire de la fonction précédente : $p = -\frac{q}{r} + \text{offset}$

2.2.11) On écrit ensuite la fonction correspondante :

```
function p = angle2step(q,offset)
% ANGLE2STEP convertit la valeur angulaire Q (rad) en une valeur de pas
% moteur P.
% OFFSET permet de spécifier à quelle valeur de pas moteur correspond la
% valeur angulaire 0.
%
% p = angle2step(q,offset);
%offset = 512( %moitié de 1024 pour avoir position 0 au centre ([-150°;+150°])

p=0;
r=(deg2rad(300))/1024; %Résolution angulaire en rad/pas

p=(-q/r) + offset;
```

Pour les deux fonctions, on oublie pas d'initialiser la variable offset dans le script principal.

```
% Paramètres de la communications série
PORT = 14; % A modifier avec le n° du port COM utilisé
OFFSET = 512; %paramètre fixe
```

2.2.12) Il est maintenant tant de mettre en pratique ces fonctions de manière concrète, en mettant en mouvement notre robot. Nous allons donc générer un mouvement, à une vitesse moyenne, et tracer cette évolution en fonction du temps.

a) Nous allons effectuer ce tracé pour différentes valeurs de consignes de vitesses.

```
time = []
position = []
speed = [50,150,250];
for i=1:length(speed)
    maquette.setStepPosition(ID_fin,165) %limite inf en step du moteur terminal
    pause(8);
    maquette.setStepPosition(ID_fin,858) %limite sup en step du moteur terminal
    tic
    maquette.setSpeed(ID_fin,speed(i))
    while (maquette.isMoving(ID_fin))
        value = maquette.getSpeed(ID_fin)
        val2speed(value)
        position = [position maquette.getStepPosition(ID_fin)];
        time = [time toc];
    end
    plot(time,position)
    drawnow
    hold on
    if i==length(speed) break
end
end
```

On choisit de créer deux listes vides 'time' et 'position' pour stocker les données de temps et de position respectivement, et on initialise une liste de vitesse. A chaque itération de la boucle for, le moteur est placé dans sa position initiale puis déplacé jusqu'à la position finale.

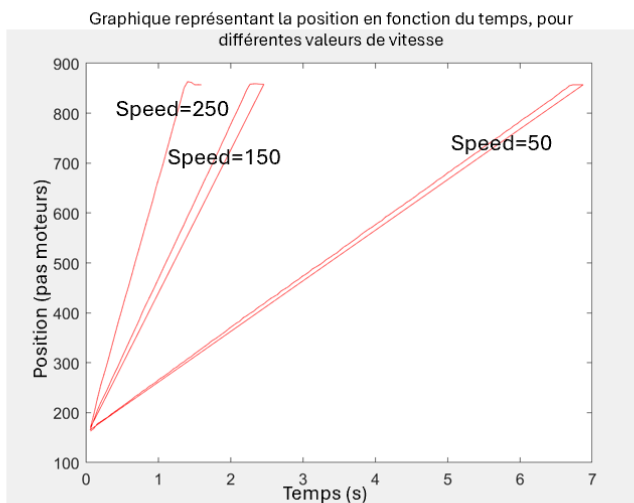


Figure 4- Tracé représentant l'évolution de la position en fonction du temps.

b) Nous allons comparer la valeur de vitesse demandée au robot dans la liste *speed* avec la vitesse réelle à laquelle il exécute le mouvement, en se basant sur les tracés en position obtenus. Pour un instant t balayant toute la durée du mouvement, soit la taille des listes *time* et *position*, la vitesse réelle correspond à l'expression suivante :

%2.2.12.b) vitesse en fonction de la position

```
v=[]

for t=1:length(position)
    v = ( position(t+1) - position(t) )/( time(t+1) - time(t) )
end
```

2.2.13) Pour mesurer directement la vitesse de rotation du moteur, on va utiliser la fonction *getSpeed*, et on va créer une fonction $v = \text{val2speed}(\text{value})$, qui va convertir la valeur brute *value* renvoyée par la fonction *getSpeed* en une valeur de vitesse v exprimée en tour/min.

La fonction *val2speed* est définie de la manière suivante :

```
function v = val2speed(value)
% VAL2SPEED convertit la valeur de vitesse VALUE lue sur le moteur en une
% valeur de vitesse V exprimée en tour/min.
%
% v = val2speed(value);

v=0;
if (value <= 1023)
    v = value*0.111
else
    v = (value - 1024)*0.111
end
```

On utilise cette fonction dans la boucle faisant varier les trois vitesses, qu'on modifie légèrement.

Le léger changement effectué, encadré ci-contre en vert, permet d'entrer autant de vitesses différentes que l'on souhaite, et adapte la boucle à la taille de la liste *speed*.

value =	value =	value =
48	148	252
v =	v =	v =
5.3280	16.4280	27.9720

2.2.14) Pour un mouvement allant de la position moteur 250 à 950, on trace l'évolution de la vitesse (en tout/min) de l'actionneur en fonction du temps. On compare ensuite cette valeur mesurée avec la valeur de consigne, qu'on entre à l'aide de la fonction *setSpeed*.

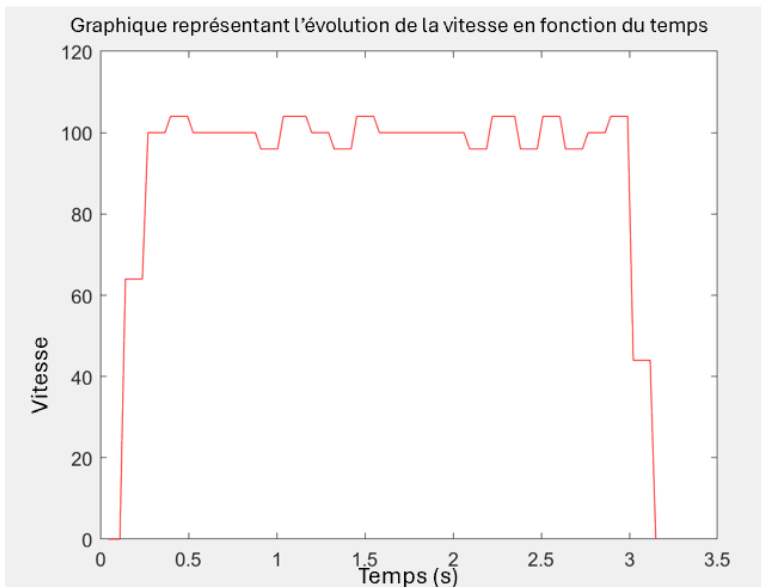
```

temps = []
vitesse = []

maquette.setStepPosition(ID_fin,250) %on règle la position initiale
pause(5); %une pause de 5s
maquette.setStepPosition(ID_fin,850) %on règle la position finale
%notre maquette ne peut pas aller
%jusqu'à 950
tic %début du chronométrage
maquette.setSpeed(ID_fin,250) %choix de la vitesse, ID correspond au 1er
%moteur, donc 1er élément de ID_list
while (maquette.isMoving(ID_fin))
    vitesse = [vitesse maquette.getSpeed(ID_fin)];
    temps = [temps toc]; %toc écrit le temps associé à la vitesse
    %à l'instant discret t
end
plot(temps,vitesse, 'red')

```

On impose une pause de 5 secondes pour que le système ait le temps d'aller à la position initiale avant de commencer le stockage des données dans les listes *temps* et *vitesse*.

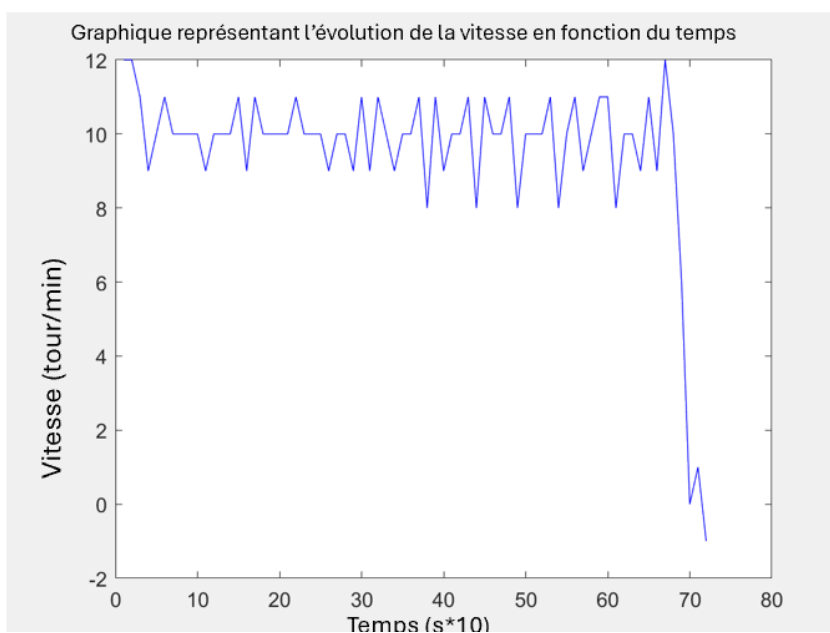


Nous observons que la vitesse est initialement instable (on observe un grand pic, qui correspond au passage de l'arrêt total à une vitesse constante). La vitesse se stabilise enfin, et finit par chuter.

Figure 5- Tracé représentant l'évolution de la vitesse en fonction du temps

2.2.15) La courbe ci-dessus permet également d'estimer la vitesse de rotation du moteur par différence finie. On va pour cela utiliser une fonction de Matlab, `diff`, afin de tracer cette vitesse.

```
f = diff(position) %fonction intrinsèque de matlab
plot(f, 'blue')
drawnow
```



On observe une allure similaire : la courbe reste plus ou moins constante, puis finit par chuter.

Figure 5bis- Tracé représentant l'évolution de la vitesse en fonction du temps

2.2.16) On souhaite maintenant évaluer la répétabilité des caractéristiques des mouvements, c'est-à-dire la vitesse et la position au cours du temps.

a) On va choisir une position initiale et une position finale, et y amener le moteur plusieurs fois de manière successive, et pour différentes vitesses.

```
time = []
position = []
repeat= 5; %nombre de répétition pour chaque vitesse
speed = [80,150,250];

for i=1:length(speed)
    for j=1:repeat %boucle pour les répétitions successives des vitesses
        maquette.setStepPosition(ID_fin,300)
        pause(5);
        maquette.setStepPosition(ID_fin,512)
        tic
        timerVal= tic;
        maquette.setSpeed(ID_fin,speed(i))
        while (maquette.isMoving(ID_fin))
            value = maquette.getSpeed(ID_fin)
            val2speed(value)
            position = [position maquette.getStepPosition(ID_fin)]; |
        end
        time = [time toc];
    end
    plot(time,position)
    %drawnow
    hold on
end
```

On choisit de répéter l'opération 5 fois, et on trace cette répétabilité.

Graphique représentant l'évolution de la position en fonction du temps, pour plusieurs vitesses.

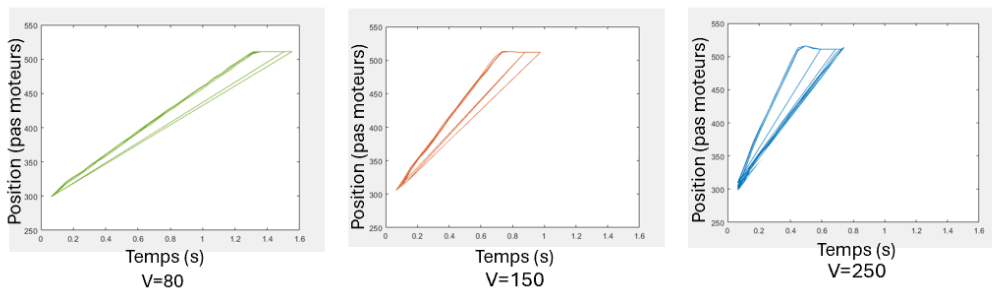


Figure 6- Tracés représentant la répétabilité de la position pour une même vitesse

b) On veut maintenant estimer la moyenne des erreurs de positions en fonction de la vitesse de déplacement utilisée. On ajoute ainsi dans notre boucle un tableau vide, qu'on appelle `err_pos`, et on va calculer la moyenne des positions finales réelles dans le but de calculer l'écart moyen entre la position finale demandée et la position.

```
err_pos = []
pos_fin = 0

pos_fin = pos_fin + position(length(position))
moy_pos_fin = pos_fin/length(pos_fin) %moyenne des positions finales réelles
err_pos = [pos_fin position(length(position))] - 512 %pour calculer l'écart moyen entre la
err_pos % position finale demandée et la position
% finale réelle
```

Utilisation du moteur en début de chaîne cinématique :

2.2.17) Nous avons vu au cours de notre TP, que le robot n'atteignait pas certaines valeurs. Nous allons donc chercher à identifier les limites angulaires de ce moteur

2.2.18) Pour cela, on va créer une fonction Matlab `new_pos=limit (pos, val_min,val_max)`, qui va contrôler que la position angulaire du robot. Si la position demandée est inférieure à la position minimale relevée, alors le robot rendra la position minimale, et pareil pour une position supérieure à la position maximale.

Les positions maximales et minimales sont relevées lorsque le robot est inactif, grâce à la commande suivante : `val_min (et val_max)= maquette.getStepPosition(ID_debut)`.

```
val_min = 798
val_max = 226
```

On rentre ensuite ces valeurs dans la fonction `new_pos`.

```
function new_pos= limit (pos, val_min, val_max)
%Assure que le robot se rend à des positions angulaire atteignables.

new_pos= zeros(1, length(pos))

for i = 1:length(pos)
    if pos(i)<= val_min %Si la position est inférieure à la position
        new_pos(i)= val_min; %minimale, on renvoie cette dernière
    elseif pos(i)>= val_max %Si la position est supérieure à la position
        new_pos(i) = val_max; % maximale, on renvoie cette dernière
    else
        new_pos(i)= pos %on donne la position demandée
    end
end
```

2.2.19) Pour un mouvement entre deux positions accessibles, nous allons :

a) tracer une nouvelle fois l'évolution de la position du moteur en fonction du temps et pour différentes valeurs de vitesse de rotation, mais cette fois ci pour le moteur en début de chaîne cinématique.

```
time = []
position = []
speed = [50,150,250];
for i=1:length(speed)
    maquette.setStepPosition(ID_debut,165) %limite inf en step du moteur terminal
    pause(8);
    maquette.setStepPosition(ID_debut,858) %limite sup en step du moteur terminal
    tic
    maquette.setSpeed(ID_debut,speed(i))
    while (maquette.isMoving(ID_debut))
        value = maquette.getSpeed(ID_debut)
        val2speed(value)
        position = [position maquette.getStepPosition(ID_debut)];
        time = [time toc];
    end
    plot(time,position)
    drawnow
    hold on
    if i==length(speed) break
end
end
```

b) On observe certaines différences avec le tracé pour le moteur terminal. En effet, on peut déjà dire que le moteur terminal est responsable du mouvement final du bras robotique , ainsi sa trajectoire est plus complexe que le moteur de début. De plus, la coordination n'est pas la même car le moteur en début de chaîne contribue à la coordination générale du robot, alors que le moteur terminal gère lui des mouvements plus précis. Ainsi, les différences observées résident surtout dans la cinématique du robot.

Bilan : Suite à notre participation au TP2, nous avons acquis la capacité de contrôler les actionneurs DYNAMIXEL qui constituent les articulations du robot fourni. Ce TP nous a permis de nous familiariser avec l'utilisation des fonctions MATLAB fournies, qui nous permettent de fixer la consigne en position des actionneurs, leur vitesse de rotation, ainsi que de mesurer leur position ou vitesse actuelle.

Nous allons à présent pouvoir coupler l'utilisation des modèles géométriques direct et inverse.

SEANCE 3 ET 4 : REALISATION EXPERIMENTALE D'UN ROBOT SERIEL

-Génération de mouvements et commande-

Objectif : L'objectif de cette séance est de maintenant de coupler les deux points évoqués précédemment, à savoir l'utilisation des modèles géométriques direct et inverse à un robot réel.

2. Exploitation expérimentale des modèles direct et inverse

2.2.1) On commence par réaliser un schéma de la cinématique du robot, afin de mieux comprendre son mécanisme.

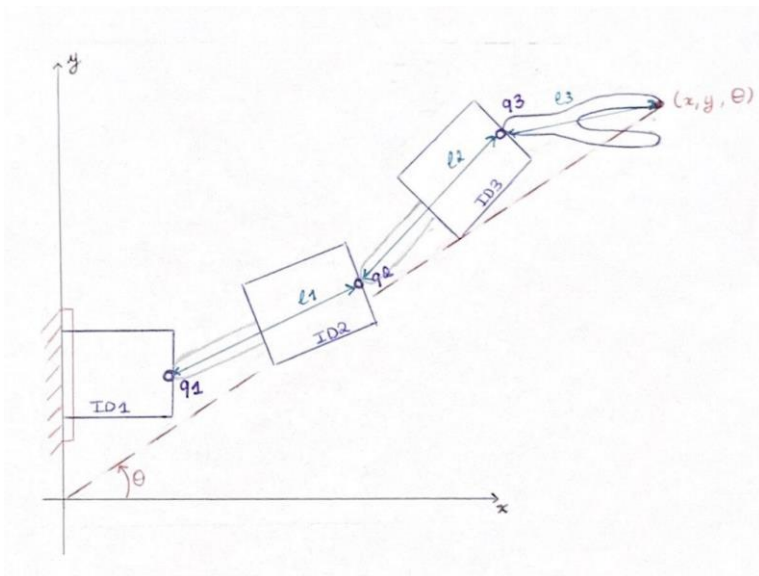


Figure 7- Schéma de la cinématique du robot (3R), dans le plan (x,y,z)

On relève différents éléments nécessaires à la bonne compréhension de ce mécanisme : tout d'abord, les ID correspondent aux trois moteurs, et nous avons rangé ces éléments dans une liste, afin de simplifier les commandes du robot. Ensuite on relève les différentes longueurs des bras du robot. Puis, on repère les 3 données angulaires, qu'on appelle q_1 , q_2 et q_3 .

2.2.2) On va maintenant pouvoir utiliser la fonction robot pour déclarer le modèle de notre robot. Comme lors du TP précédent, on oublie pas d'initialiser les paramètres qui seront nécessaires pour assurer le lien entre le robot et la machine :

```
maquette = robot('real',PORT,1000000);
maquette.setSpeed(ID_list, [100 100 100])

maquette.setStepPosition(ID_list, q_step_home) %on pose les positions initiales/ angles en step (idéales)
while any(maquette.isMoving(ID_list))
    pause(0.02)
end
q_step_real = maquette.getStepPosition(ID_list); %pour obtenir les positions initiales/ angles en step REELLES
q_rad_real = step2angle(q_step_real,OFFSET); %conversion angle en step à angle en rad

%%

%on initialise le modèle

modele = robot('model', segment);

modele.setAngularPosition(q_rad_real); %on synchronise les positions du robot avec le modèle pour pouvoir
% utiliser à la fois les fonctions de la maquette et du modèle

%modele.setAngularPosition([0 0 0]);
modele.plot(0, 'red', 0)
```

On note ici l'importance de créer des listes contenant nos variables : comme dit précédemment, la commande du robot est grandement simplifiée. En effet, il suffira d'entrer une seule fois les ID pour pouvoir les utiliser naturellement tout au long du programme.

2.2.3) Nous allons utiliser la fonction mod_geo_dir, afin de déterminer la pose (x,y,θ) de l'organe terminal pour 3 configurations articulaires q_1 , q_2 , q_3 .

```
%CONFIGURATION 1
q_step_ini = [400 400 220] %pos ini
maquette.setStepPosition(ID_list,q_step_ini ); %initialisation position ini ligne
q_step_1 = maquette.getStepPosition(ID_list)
q_angle_1 = step2angle(q_step_1,OFFSET); %on récupère les positions en radians pour le modele
grad = deg2rad(q_angle_1);
modele.setAngularPosition(grad);
```

Le robot fonctionnant en pas moteur, on initialise q_step_ini , qui sera notre position, et que l'on récupère en radians à l'aide de la fonction step2angle créée précédemment.

Pour récupérer la pose de l'organe terminal, on définit la configuration géométrique directe du robot, que l'on va stocker dans T. On calcule ensuite l'angle d'orientation du robot θ_q , et on utilise $T(1,4)$ et $T(2,4)$ qui représentent les composantes de translation x et y.

```
T = modele.mod_geo_dir()

theta_q = rad2deg(atan2 (T(2,1), T(1,1)));
theta_q = inv_rotation_Z(T(1:3,1:3));

q_to_x = T(1, 4);
q_to_y = T(2, 4);
```

Nous avons pris soin avant chaque utilisation du robot de relever ses positions limites. En effet, lorsque le robot est inactif, nous pouvons bouger ses bras aisément. On se rappelle de la fonction `new_pos`, créée lors de la séance 2.

2.2.4) Au cours de nos TP, nous avons pu remarquer que nous ne maîtrisons pas complètement sa trajectoire dans l'espace opérationnel. En effet, le robot réel est en fait limité par certains paramètres, comme ses bras ou encore sa fixation au bâti. Ainsi, même si le modèle du robot virtuel peut simuler parfaitement une rotation parfaite, le robot ne peut quant à lui pas vraiment tourner à 360° : on parle alors d'« Invalid Angle ».

Cette limite n'est pas à négliger, car dans le cas inverse, cela peut bloquer le robot, qui va alors nécessiter un redémarrage, d'où l'importance de la fonction `new_pos`.

2.2.5) Il est maintenant temps de décider d'une pose (x,y,θ) cible dans l'espace opérationnel, et déterminer les configurations articulaires q permettant d'atteindre cette pose. Pour réaliser cela, nous allons utiliser la fonction `mod_geo_inv`, explicitée plus haut.

```
x=.15
y=.1
theta_z = pi/4

[q1, q2] = modele.mod_geo_inv(x,y,theta_z)
```

Comme lors de la séance 1 on note deux solutions dues à la symétrie de l'espace opérationnel. Elles correspondent aux deux configurations possibles pour atteindre la cible donnée. Il s'agit de deux couples d'angles des deux premiers actionneurs.

2.2.6) Grâce à Matlab, nous pouvons analyser le robot pendant qu'il bouge. En effet, à chaque mouvement du robot, le modèle de robot que nous avons initialisé bouge en même temps.

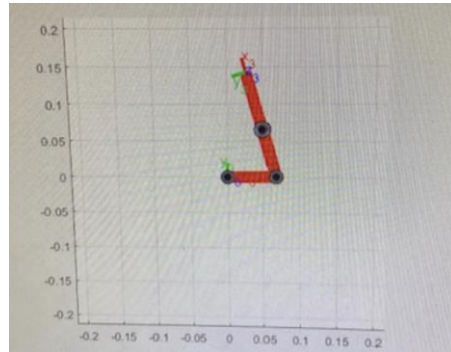
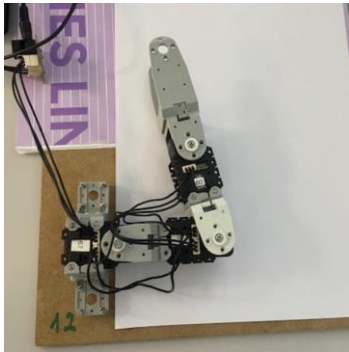


Figure 8 : Comparaison entre le robot 'réel' et la robot modélisé

3) Génération de trajectoires

Notre but est maintenant de faire en sorte que le robot soit capable de suivre une trajectoire exprimée dans l'espace opérationnel, et ce en terme de positions et d'orientations que l'organe terminal du bras devra atteindre au cours du temps.

Nous allons réaliser cela par interpolation. Ici, nous allons nous pencher sur une interpolation exprimée dans l'espace opérationnel uniquement. On va alors pouvoir déterminer par interpolation linéaire un certain nombre de configurations intermédiaires, partant de (x^o, y^o, θ^o) et menant en (x, y, θ)

Nous allons donc créer une boucle qui va effectuer une interpolation articulaire en rotation des positions angulaires entre la position initiale et la position finale sur un nombre fini d'itérations (défini par N).

Après avoir initialiser les positions angulaires, nous définissons la vitesse de déplacement pour chaque moteur, puis positionne la bras robotique à sa position initiale.

```
q_rad_ini = q_rad_real;
q_rad_fin = [pi/12, 0, 0];

maquette.setSpeed(ID_list, [160, 160, 160])
maquette.setStepPosition(ID_list, q_step_home)
```

avec

```
q_step_real = maquette.getStepPosition(ID_list); %pour obtenir les positions initiales/ angles en step REELLES
q_rad_real = step2angle(q_step_real, OFFSET); %conversion angle en step à angle en rad
```

Puis initialise une boucle à travers chaque itération de l'interpolation.

```

N=100;
for n=0:N
    q_rad_des = q_rad_ini + n/N .* (q_rad_fin - q_rad_ini)
    q_step_des = angle2step(q_rad_des,OFFSET);

    maquette.setStepPosition(ID_list, q_step_des);
    while any(maquette.isMoving(ID_list))
        pause(0.0001)
    end

    q_step_real = maquette.getStepPosition(ID_list);
    q_rad_real = step2angle(q_step_real,OFFSET);

    modele.setAngularPosition(q_rad_real);
    modele.plot(0, 'red' , 0)
    drawnow
end

return

```

Pour programmer au modèle la position cible, nous utilisons le modèle géométrique inverse.

L'objectif désormais est de réaliser des trajectoires par interpolation linéaire entre deux configurations opérationnelles.

La première étape consiste à faire tracer au robot une ligne droite. On ne procède donc pas à l'interpolation articulaire, qui permet seulement de parcourir les positions angulaires des actionneurs. En utilisant l'interpolation opérationnelle, on entre les coordonnées cibles de départ et d'arrivée en repère cartésien (x, y) et/ou θ (mais nous avons remarqué qu'en entrant x et y, l'entrée de θ n'était pas indispensable). Le robot adapte ensuite la rotation de ses actionneurs pour suivre la trajectoire linéaire qu'on lui demande d'exécuter. Il n'y a donc pas de courbure.

Initialisation des données

```

% 1ere étape : la barre verticale
theta_ini= pi/6
theta_fin = pi/4

x_ini= segment(1) + 0.01    % position arbitraire mais stratégique pour
x_fin= x_ini                % s'assurer qu'elle appartient à
                           % l'espace opérationnel

y_ini= 0.15                 %Idem commentaire précédent
y_fin= y_ini - 0.05

X= [x_ini y_ini theta_ini]  % On entre nos valeurs dans une liste en
Xfin = [x_fin y_fin theta_fin] % dissociant les informations de départ et
                           % d'arrivée

```

Interpolation linéaire


```

N_2=10
for n=1:(N_2)
    n_2 =n-1
    x_inter(n) = x_ini + (x_fin - x_ini)*(n_2/N_2)
    y_inter(n) = y_ini + (y_fin - y_ini)*(n_2/N_2)
    theta_inter(n) = theta_ini + (theta_fin - theta_ini)*(n_2/N_2)
    [q1, q2] = modele.mod_geo_inv(x_inter(n),y_inter(n),theta_inter(n))
    %on choisit arbitrairement la configuration q1

    q1_step = angle2step(q1,OFFSET);      %conversion q1 de radian en step
    maquette.setStepPosition(ID_list, q1_step) %pour que la maquette aille
    modele.setAngularPosition(q1);          %à ce point
    modele.plot(0, 'red' , 0)
    pause(.5)
end

```

On a choisi $n=10$ configurations intermédiaires, qu'on a nommé ci-dessus N_2 .

Comme commenté ci-dessus, on a la configuration q_1 qui est exprimée en radian, on peut l'entrer ainsi naturellement dans les fonctions du modèle. Cependant pour l'utiliser avec les fonctions de la maquette, il nous faut la convertir en *step* à l'aide de la fonction *angle2step*.

En visionnant la vidéo que nous avons pris, vous constaterez que le robot effectue bien le mouvement désiré. En augmentant le nombre de configuration intermédiaires, nous pensions avoir un mouvement moins saccadé et donc plus souple, mais ce n'était pas le cas : au contraire les saccades étaient plus fortes.

Nous avons ainsi pu tracer différentes figures.

PARTIE BONUS

Nous avons réussi à faire des formes plus complexes, tels que les lettres L et N

Écriture de la lettre L

La première étape est identique à la barre verticale expliquée précédemment. On passe ainsi l'étape la succède, soit la barre horizontale.

```

% 2e étape : la barre horizontale

theta_ini_2= theta_fin
theta_fin_2= theta_fin
x_ini_2= x_fin
x_fin_2 = x_fin +0.02

%y_ini= l_tot*sin(theta_ini)
%y_fin = 0.5*y_ini
y_ini_2= y_fin
y_fin_2= y_fin

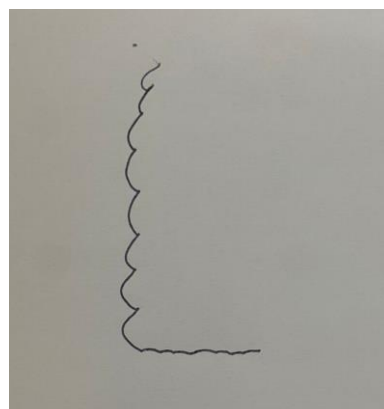
X2= [x_ini_2 y_ini_2 theta_ini]
Xfin2 = [x_fin_2 y_fin_2 theta_fin]

pause(3)
N_2=10
for n=1:(N_2)
    n_2 =n-1
    x_inter(n) = x_ini_2 + (x_fin_2 - x_ini_2)*(n_2/N_2)
    y_inter(n) = y_ini_2 + (y_fin_2 - y_ini_2)*(n_2/N_2)
    theta_inter(n) = theta_ini_2 + (theta_fin_2 - theta_ini_2)*(n_2/N_2)
    [q1, q2] = modele.mod_geo_inv(x_inter(n),y_inter(n),theta_inter(n))

    q1_step = angle2step(q1,OFFSET);
    maquette.setStepPosition(ID_list, q1_step)
    modele.setAngularPosition(q1);
    modele.plot(0, 'red' , 0)
    pause(.5)
end

```

Voici le tracé obtenu :



Écriture de la lettre N

La première étape est identique à la barre verticale expliquée précédemment. On passe ainsi l'étape la succède, soit la barre horizontale.

Les étapes pour écrire la lettre N sont similaires à L, hormis la diagonale de cette dernière. On commence par une barre allant de bas en haut, ensuite la diagonale vers la droite vers le bas puis une dernière barre de nouveau de bas en haut.

```
%écriture de la lettre "N" - REUSSI
```

```
% 1ere étape : la barre verticale inverse
```

```
x_ini= segment(1) + 0.01  
x_fin= x_ini
```

```
y_ini= 0.10  
y_fin= 0.15
```

```
X= [x_ini y_ini theta_ini]  
Xfin = [x_fin y_fin theta_fin]
```

```
%maquette.setStepPosition(ID_list, q_step_home);  
pause(3)
```

```
N_2=10
```

```
for n=1:(N_2)  
    n_2 =n-1  
    x_inter(n) = x_ini + (x_fin - x_ini)*(n_2/N_2)  
    y_inter(n) = y_ini + (y_fin - y_ini)*(n_2/N_2)  
    theta_inter(n) = theta_ini + (theta_fin - theta_ini)*(n_2/N_2)  
    [q1, q2] = modele.mod_geo_inv(x_inter(n),y_inter(n),theta_inter(n))
```

```
    q1_step = angle2step(q1,OFFSET);  
    maquette.setStepPosition(ID_list, q1_step)  
    modele.setAngularPosition(q1);  
    modele.plot(0, 'red' , 0)  
    pause(.5)
```

```
end
```

```
% 2e étape : la barre diagonale
```

```
x_ini_2= x_fin  
x_fin_2 = x_ini +0.03
```

```
%y_ini= 1_tot*sin(theta_ini)  
%y_fin = 0.5*y_ini
```

```
y_ini_2= y_fin  
y_fin_2= y_ini
```

```
X2= [x_ini_2 y_ini_2 theta_ini]  
Xfin2 = [x_fin_2 y_fin_2 theta_fin]
```

```
pause(3)  
N_2=10
```

```
for n=1:(N_2)  
    n_2 =n-1  
    x_inter(n) = x_ini_2 + (x_fin_2 - x_ini_2)*(n_2/N_2)  
    y_inter(n) = y_ini_2 + (y_fin_2 - y_ini_2)*(n_2/N_2)  
    theta_inter(n) = theta_ini_2 + (theta_fin_2 - theta_ini_2)*(n_2/N_2)  
    [q1, q2] = modele.mod_geo_inv(x_inter(n),y_inter(n),theta_inter(n))
```

```
    q1_step = angle2step(q1,OFFSET);  
    maquette.setStepPosition(ID_list, q1_step)  
    modele.setAngularPosition(q1);  
    modele.plot(0, 'red' , 0)  
    pause(.5)
```

```
end
```

```
% 3e étape : la dernière barre verticale
```

```
x_ini_3= x_fin_2  
x_fin_3 = x_fin_2
```

```
y_ini_3= y_ini  
y_fin_3= y_fin
```

```
X= [x_ini y_ini theta_ini]  
Xfin = [x_fin y_fin theta_fin]
```

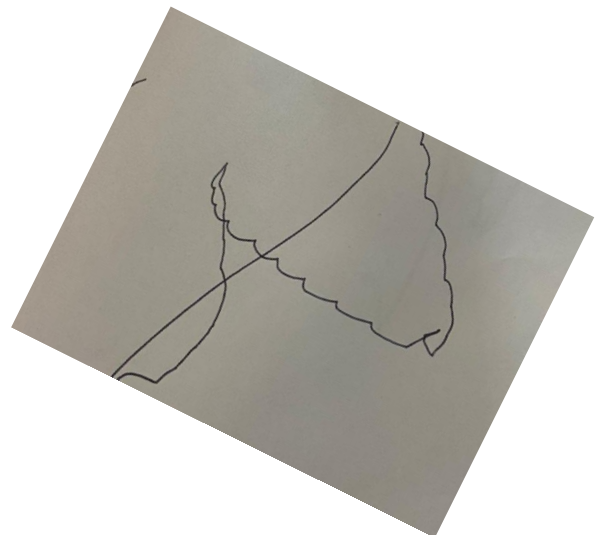
```
%maquette.setStepPosition(ID_list, q_step_home);  
pause(3)
```

```
N_2=10
```

```
for n=1:(N_2)  
    n_2 =n-1  
    x_inter(n) = x_ini_3 + (x_fin_3 - x_ini_3)*(n_2/N_2)  
    y_inter(n) = y_ini_3 + (y_fin_3 - y_ini_3)*(n_2/N_2)  
    theta_inter(n) = theta_ini + (theta_fin - theta_ini)*(n_2/N_2)  
    [q1, q2] = modele.mod_geo_inv(x_inter(n),y_inter(n),theta_inter(n))
```

```
    q1_step = angle2step(q1,OFFSET);  
    maquette.setStepPosition(ID_list, q1_step)  
    modele.setAngularPosition(q1);  
    modele.plot(0, 'red' , 0)  
    pause(.5)
```

```
end
```



On observe sur le tracé une ligne partant de l'une des pointes, vers l'autre pointe.

Nous avons tenté de dessiner également un cercle mais nos tentatives n'étaient pas satisfaisantes. Pour le cercle, nous avons essayé plusieurs pistes. Nous avons pensé à le faire à l'aide de l'interpolation articulaire qui atteint une position d'arrivée depuis une position de départ de manière circulaire et non linéaire. Cette piste n'a pas fonctionné.

Nous avons ensuite pensé à faire varier l'angle initial θ de manière constante à chaque itération mais nous n'avons pas pu aller au bout de cette idée.

Puis nous avons pensé à utiliser q_1 pour dessiner la moitié d'un cercle, de gauche à droite, puis utiliser q_2 pour l'autre moitié, puisque ce qui nous limitait était les limites des actionneurs.

Que faire contre les saccades ?

Pour éviter les saccades observées dans les implémentations précédentes, on va essayer d'utiliser le Jacobien dans la planification et le contrôle des mouvements du robot.

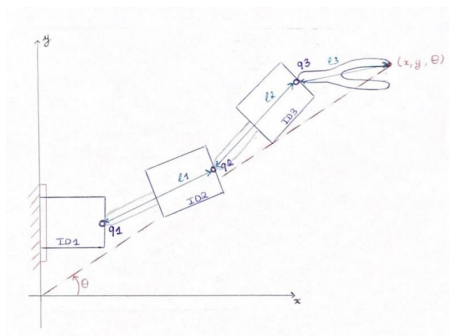
On commence par fixer $\theta_3=0$, pour pouvoir traiter le robot comme un 2R. Dans ce cas, les 2 degrés de liberté sont les coordonnées x et y , à commander à travers les moteurs q_1 et q_2 .

On va dans un premier temps, écrire le Jacobien du robot 2R :

$$J = \begin{bmatrix} \frac{\partial x}{\partial q_1} & \frac{\partial x}{\partial q_2} \\ \frac{\partial y}{\partial q_1} & \frac{\partial y}{\partial q_2} \end{bmatrix}$$

Avec x et y correspondant aux coordonnées de l'effecteur final dans un système de coordonnées cartésiennes.

On peut maintenant calculer les positions x , y , θ . On reprend la figure 7.



$$x = l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2)$$

$$y = l_1 \sin(\theta_1) + l_2 \sin(\theta_1 + \theta_2)$$

On cherche maintenant à exprimer les vitesses, grâce auxquelles nous exprimerons la matrice Jacobienne.

$$\begin{aligned}
 \dot{x} &= -l_1 \dot{\theta}_1 \sin(\theta_1) - l_2 \dot{\theta}_{12} \sin(\theta_{12}) \\
 \dot{y} &= l_1 \dot{\theta}_1 \cos(\theta_1) + l_2 \dot{\theta}_{12} \cos(\theta_{12}) \\
 \dot{\theta} &= \dot{\theta}_1 + \dot{\theta}_2
 \end{aligned}
 \quad \Rightarrow \quad
 J = \begin{bmatrix} -l_1 \sin(\theta_1) - l_2 \sin(\theta_1 + \theta_2) & -l_2 \sin(\theta_1 + \theta_2) \\ l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2) & l_2 \cos(\theta_1 + \theta_2) \end{bmatrix}$$

CONCLUSION :

Dans ces séances de manipulation de robots sériels, nous avons exploré divers aspects de la modélisation, de la commande et de la génération de mouvements. Nous avons commencé par la modélisation géométrique directe et inverse, permettant de calculer les positions et orientations du bras robotique en fonction des paramètres articulaires, ainsi que de déterminer les paramètres articulaires nécessaires pour atteindre une position et une orientation spécifique. Ensuite, nous avons caractérisé les actionneurs du système et exploré leurs limitations, notamment en termes de vitesses et de positions angulaires. Enfin, nous avons abordé la génération de trajectoires par interpolation, en nous concentrant sur l'interpolation articulaire et opérationnelle.

Nous avons constaté les limites du modèle virtuel en le comparant aux mouvements réels du robot. Nous avons rencontré diverses contraintes physiques, telles que la difficulté à reproduire précisément une commande de position ou les butées articulaires qui peuvent limiter le robot dans une position non désirée. De plus, il a été difficile de commander des trajectoires précises et complexes. Malgré une simulation fluide dans le modèle virtuel, la version réelle du robot présentait des mouvements saccadés ou imprévisibles lors du passage d'un point à un autre de la trajectoire. En conclusion, ce projet a été très enrichissant sur le plan pédagogique. Nous avons apprécié l'opportunité d'observer nos réalisations dans le domaine de la robotique, de développer notre propre programmation et de l'implémenter progressivement au fil des séances de travaux pratiques.